Pivot Calibration

Luke Doladille

October 19, 2025

Introduction on pivot calibration

Intra-operative imaging requires tool tracking in order to ensure the accurate mapping of tools relative to real-time imaging data. In order to determine the position of the tool tip (P_t, P_p), we rely on a calibration process called pivot calibration.

We use the correspondence $Fi(R_i, t_i)$ between the coordinate systems of two objects: the marker (local coordinate system, LCS), physically attached to the rotating tool, and the tracker (global coordinate system, GCS), which serves as the reference point for the camera.

A transformation from a point P_t in the marker's coordinate system to its corresponding point P_p in the tracker's coordinate system is given by:

$$P_p = R_i \cdot P_t + t_i$$

Where:

- R_i is the 3×3 rotation matrix, i.e., the matrix which, when multiplied by the three base vectors of the global coordinate system, outputs the coordinates of the LCS base vectors in the global coordinate system ;
- t_i is the 3×1 translation vector, i.e., the coordinates of the origin of the local coordinate system in the global coordinate system.

R_i and t_i are combined into a 3×4 matrix representing a **rigid body** transformation.

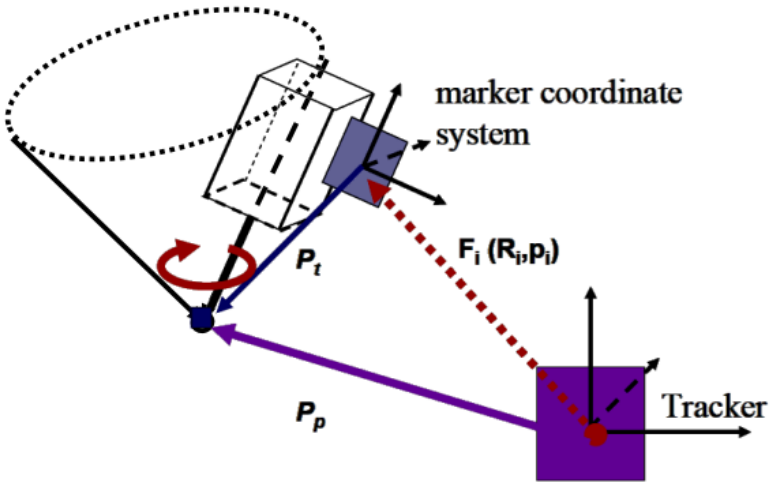


Figure 1: Principles and notations of pivot calibration. t_i is p_i in the picture.

Image source: Course content from BME8113-Advanced Intraoperative Imaging.

Task 1: Compute the coordinate of the tool tip in the pointer's local coordinate system

The objective of this task is to find P_t, the position of the pointer's tip in the pointer's local coordinate system (LCS). The pivot calibration's principle is that the operator rotates the pointer in a circular motion while maintaining the pointer's tip into place. This means P_p and P_t are constant.

For each measurement, we are given R_i and t_i in the form of a 3×4 matrix. We have:

$$P_p = R_i \cdot P_t + t_i$$

Since both P_p and P_t are unknown but constant, we can rearrange the equation to group the unknowns on one side and the knowns on the other:

$$R_i \cdot P_t - P_p = -t_i$$

This is a system of 3 linear equations for each measurement. With N measurements, we can stack them all to form a large, overdetermined system of linear equations of the form $A\mathbf{x} = \mathbf{b}$. We note:

$$\mathbf{x} = \begin{pmatrix} P_t \\ P_p \end{pmatrix}$$

This is a 6×1 vector. The matrix A and vector $\mathbf{b}$ are constructed by stacking the components from each measurement $i = 1...N$:

$$A = \begin{pmatrix} R_1 & -I \\ R_2 & -I \\ \vdots & \vdots \\ R_N & -I \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} -t_1 \\ -t_2 \\ \vdots \\ -t_N \end{pmatrix}$$

where I is the 3×3 identity matrix. The resulting matrix A will be $(3N \times 6)$ and vector $\mathbf{b}$ will be $(3N \times 1)$. We find the best-fit solution using the least-squares method:

$$\mathbf{x} = (A^T A)^{-1} A^T \mathbf{b}$$

This gives us the 6×1 vector $\mathbf{x}$, from which we extract P_t as the first 3 components.

Task 2: Calculation of Root Mean Square Error (RMSE)

The Root Mean Square Error (RMSE) is computed to quantify the quality and consistency of the pivot calibration. This metric represents the average residual error between the predicted pivot point position for each measurement and the single, best-fit pivot point P_p derived from the complete dataset.

The least-squares solution provides the optimal coordinates for the pointer's tip in the LCS, P_t, and the pointer's tip in the camera's coordinate system, P_p. To calculate the RMSE, the pivot point's position in the camera frame is first re-calculated for each individual measurement i, using the known local tool tip vector P_t and the i-th transformation matrix $[R_i|t_i]$. This predicted position is denoted $\hat{P}_{p,i}$:

$$\hat{P}_{p,i} = R_i \cdot P_t + t_i$$

The residual error e_i for each measurement is the Euclidean distance between this predicted position $\hat{P}_{p,i}$ and the globally optimized pivot position P_p:

$$e_i = \|\hat{P}_{p,i} - P_p\|$$

Finally, the RMSE is calculated as the square root of the mean of these squared residual errors over all N measurements:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{P}_{p,i} - P_p\|^2}$$

A low RMSE (typically sub-millimeter) indicates high stability of the pivot point during data acquisition and, consequently, a high-quality and reliable calibration.

Implementation in Python

```python
import numpy as np
import ast

def solve_pivot_calibration(filename="Tpointer2Cam.txt"):

    R_matrices = []
    t_vectors = []
    A_list = []
    b_list = []
    I_neg = -np.eye(3) # The -I block

    print(f"Reading data from '{filename}'...")

    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()
            if not line: # Skip empty lines
                continue

            # Safely evaluate the string "[[-0.9...], ...]"
            T_i = np.array(ast.literal_eval(line))

            if T_i.shape != (3, 4):
                print(f"Warning: Skipping malformed line. Shape was {T_i.shape}"
    )
                continue

            R_i = T_i[:, :3]
            t_i = T_i[:, 3]

            # Store for RMSE calculation
            R_matrices.append(R_i)
            t_vectors.append(t_i)

            # Build the blocks for the Ax = b system
            # [R_i  -I] * [P_t] = [-t_i]
            #             [P_p]
            A_list.append(np.hstack([R_i, I_neg]))
            b_list.append(-t_i)

    if not A_list:
        print("Error: No valid data was read from the file.")
        return

    # --- Task 1: Solve for Tip Coordinate ---

    A = np.vstack(A_list)
    b = np.concatenate(b_list)
    num_measurements = len(R_matrices)
```

```
49
50      print(f"Data loaded. Found {num_measurements} measurements.")
51
52      # Solve the overdetermined system Ax = b
53      solution, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
54
55      P_t = solution[0:3] # First 3 elements are P_t (local tip)
56      P_p = solution[3:6] # Last 3 elements are P_p (pivot in cam)
57
58      print("\n--- Task 1 Result ---")
59      print(f"Computed tool tip coordinate in pointer LCS (P_t):")
60      print(f"  x = {P_t[0]:.4f}")
61      print(f"  y = {P_t[1]:.4f}")
62      print(f"  z = {P_t[2]:.4f}")
63
64      # --- Task 2: Compute RMSE ---
65
66      sum_squared_errors = 0
67      for i in range(num_measurements):
68          R_i = R_matrices[i]
69          t_i = t_vectors[i]
70
71          # Predict pivot point in camera space using this measurement's
72          # transform and our calculated P_t
73          P_p_predicted_i = R_i @ P_t + t_i
74
75          # Calculate the error
76          e_i_vec = P_p_predicted_i - P_p
77          squared_error = np.dot(e_i_vec, e_i_vec)
78          sum_squared_errors += squared_error
79
80      mean_squared_error = sum_squared_errors / num_measurements
81      rmse = np.sqrt(mean_squared_error)
82
83      print("\n--- Task 2 Result ---")
84      print(f"Root Mean Square Error (RMSE) of calibration: {rmse:.4f} mm")
85
86  solve_pivot_calibration("Tpointer2Cam.txt")
```

Results

```
Reading data from 'Tpointer2Cam.txt'...
Data loaded. Found 259 measurements.

--- Task 1 Result ---
Computed tool tip coordinate in pointer LCS (P_t):
  x = 208.4182
  y = -0.6433
  z = -33.1314

--- Task 2 Result ---
Root Mean Square Error (RMSE) of calibration: 0.4482 mm
```